

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2

По дисциплине: «Естественно-языковой интерфейс ИС»

Тема: «Разработка системы автоматического реферирования документов»

Выполнил:

Студент 4 курса

Группы ИИ-23

Романюк А. П.

Проверил:

Булей Е. В.

Ход работы

Цель работы – освоить на практике основные принципы автоматического реферирования документов.

на входе – на входе – текстовые документы одинакового размера (например, 10 страниц формата A4), содержащие тексты из предметных областей на естественных языках согласно варианту подлежащие процедуре автоматического реферирования;

на выходе – активная ссылка на исходный документ и построенный, в соответствии с вариантом реферат документа, состоящий из 2-х разделов: - 1 - классического реферата и реферата в виде списка ключевых слов (по методу Sentence extraction); -2 – реферата, построенного с применением машинного обучения (ML).

наличие средств сохранения в файл и распечатки полученной на выходе информации;

интерфейс системы должен быть предельно простым и доступным для пользователей любого уровня, содержать понятный набор инструментов и средств, а также help-средства.

9	Испанский, Английский	Sentence extraction+ ML	Научные статьи по computer science, Сочинения по литературе
---	--------------------------	----------------------------	--

Код программы:

```
import re
import nltk
import tkinter as tk
from tkinter import filedialog, Text
from sklearn.feature_extraction.text import TfidfVectorizer
from transformers import T5ForConditionalGeneration, T5Tokenizer
from docx import Document
import threading

nltk.download('punkt', quiet=True)
nltk.download('stopwords', quiet=True)
nltk.download("punkt_tab")
from nltk.corpus import stopwords
from nltk.tokenize import sent_tokenize, word_tokenize

def extract_text_from_docx(docx_file):
    doc = Document(docx_file)
    full_text = []
    for paragraph in doc.paragraphs:
        full_text.append(paragraph.text)
    return '\n'.join(full_text)

def clean_text(text):
    text = re.sub(r'\s+', ' ', text)
    text = re.sub(r'\d+', '', text)
    return text

def extract_sentences_tfidf(text):
    sentences = sent_tokenize(text)
    stop_words = set(stopwords.words('english'))

    cleaned_sentences = [
        ' '.join([word for word in word_tokenize(sentence.lower()) if word.isalpha() and word not in stop_words]) for
        sentence in sentences]

    vectorizer = TfidfVectorizer()
    tfidf_matrix = vectorizer.fit_transform(cleaned_sentences)

    return sentences, tfidf_matrix.sum(axis=1)

def generate_summary(sentences, sentence_weights, summary_length=10):
    ranked_sentences = sorted(((weight, sentence) for sentence, weight in zip(sentences, sentence_weights)),
```

```

        reverse=True)
summary = ''.join([sentence for weight, sentence in ranked_sentences[:summary_length]])
return summary

def generate_ml_summary(text):
    model_name = "t5-base" # Вы можете использовать "t5-small" или "t5-large" в зависимости от ваших ресурсов
    try:
        model = T5ForConditionalGeneration.from_pretrained(model_name)
        tokenizer = T5Tokenizer.from_pretrained(model_name)

        input_text = "summarize: " + text
        inputs = tokenizer(input_text, return_tensors="pt", max_length=512, truncation=True)
        summary_ids = model.generate(inputs["input_ids"], max_length=150, min_length=40, length_penalty=2.0,
                                     num_beams=4, early_stopping=True)
        summary = tokenizer.decode(summary_ids[0], skip_special_tokens=True)
        return summary
    except Exception as e:
        print(e)
        return f"Ошибка при создании ML-сводки: {str(e)}"

def summarize_docx(docx_file, method):
    text = extract_text_from_docx(docx_file)
    cleaned_text = clean_text(text)
    if method == "TF-IDF":
        sentences, sentence_weights = extract_sentences_tfidf(cleaned_text)
        summary = generate_summary(sentences, sentence_weights)
    elif method == "ML":
        summary = generate_ml_summary(cleaned_text)
    return text, summary

def open_file():
    filepath = filedialog.askopenfilename(filetypes=[("Document files", "*.docx")])
    if filepath:
        selected_file.set(filepath)
        text = extract_text_from_docx(filepath)
        original_text_box.delete(1.0, tk.END)
        original_text_box.insert(tk.END, text)

def run_summarize():
    threading.Thread(target=summarize).start()

def summarize():
    if selected_file.get():
        method_choice = method.get()
        original_text_box.delete(1.0, tk.END)
        summary_text_box.delete(1.0, tk.END)
        try:
            original_text, summary = summarize_docx(selected_file.get(), method_choice)
            original_text_box.insert(tk.END, original_text)
            summary_text_box.insert(tk.END, summary)
        except Exception as e:
            summary_text_box.insert(tk.END, f"Ошибка при создании сводки: {str(e)}")

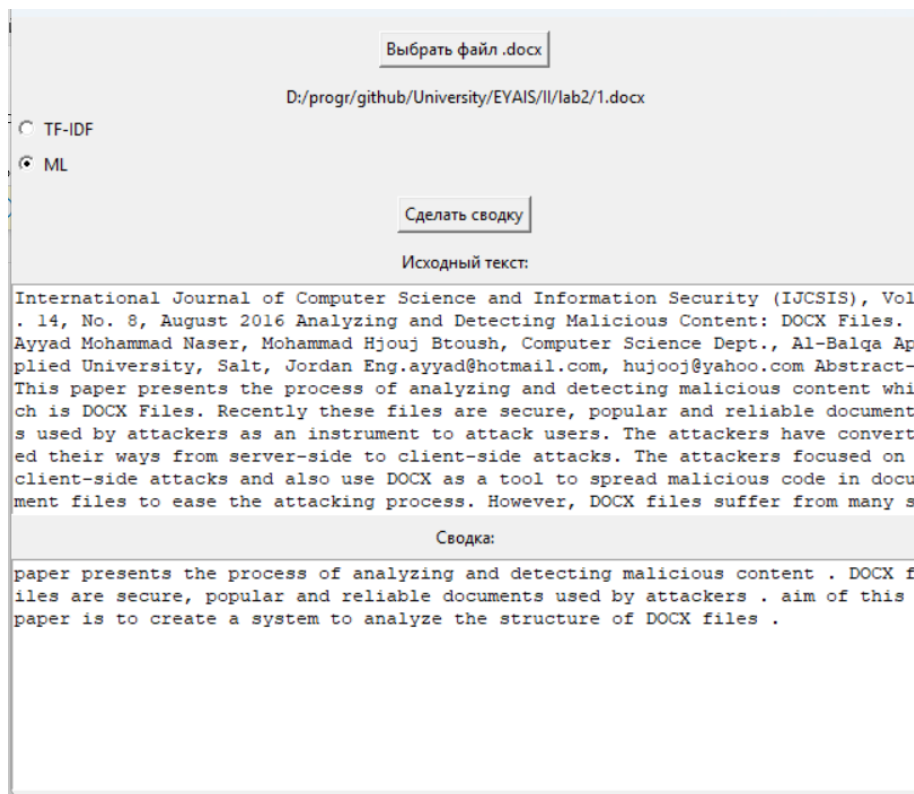
root = tk.Tk()
root.title("сокращаем")
selected_file = tk.StringVar()
method = tk.StringVar(value="TF-IDF")
file_button = tk.Button(root, text="Выбрать файл .docx", command=open_file)
file_button.pack(pady=10)
file_label = tk.Label(root, textvariable=selected_file)
file_label.pack()
tfidf_radio = tk.Radiobutton(root, text="TF-IDF", variable=method, value="TF-IDF")
tfidf_radio.pack(anchor="w")
ml_radio = tk.Radiobutton(root, text="ML", variable=method, value="ML")
ml_radio.pack(anchor="w")
summarize_button = tk.Button(root, text="Сделать сводку", command=run_summarize)
summarize_button.pack(pady=10)

```

```

original_text_label = tk.Label(root, text="Исходный текст:")
original_text_label.pack()
original_text_box = Text(root, height=10, width=80)
original_text_box.pack(pady=5)
summary_text_label = tk.Label(root, text="Сводка:")
summary_text_label.pack()
summary_text_box = Text(root, height=10, width=80)
summary_text_box.pack(pady=5)
root.mainloop()

```



Вывод: в ходе выполнения лабораторной работы освоил принципы разработки системы автоматического реферирования документов